

一.线程属性设置的思路步骤

1.线程的属性有哪些

线程属性结构如下:

```
typedef struct
{
    int                detachstate;  线程的分离状态
    int                schedpolicy;  线程调度策略
    struct sched_param schedparam;  线程的调度参数
    int                inheritsched; 线程的继承性
    int                scope;        线程的作用域
    size_t             guardsize;  线程栈末尾的警戒缓冲区大小
    int                stackaddr_set;
    void *             stackaddr;   线程栈的位置
    size_t             stacksize;   线程栈的大小
}pthread_attr_t;
```

2.如何设置线程的属性

第一步: 定义线程属性结构体变量, 并初始化

```
pthread_attr_t myattr;
int pthread_attr_init(pthread_attr_t *attr);
```

第二步: 调用linux系统提供的设置线程属性的函数

```
pthread_attr_setxxxx()
```

API	功能
pthread_attr_destroy()	销毁线程属性
pthread_attr_getaffinity_np()	获取 CPU 亲和度
pthread_attr_getdetachstate()	获取分离属性
pthread_attr_getguardsize()	获取栈警戒区大小
pthread_attr_getinheritsched()	获取继承策略
pthread_attr_getschedparam()	获取调度参数
pthread_attr_getschedpolicy()	获取调度策略
pthread_attr_getscope()	获取竞争范围
pthread_attr_getstack()	获取栈指针和栈大小
pthread_attr_getstackaddr()	已弃用
pthread_attr_getstacksize()	获取栈大小
pthread_attr_init()	初始化线程属性
pthread_attr_setaffinity_np()	设置 CPU 亲和度
pthread_attr_setdetachstate()	设置分离属性
pthread_attr_setguardsize()	设置栈警戒区大小
pthread_attr_setinheritsched()	设置继承策略
pthread_attr_setschedparam()	设置调度参数
pthread_attr_setschedpolicy()	设置调度策略
pthread_attr_setscope()	设置竞争范围
pthread_attr_setstack()	设置栈的位置和栈大小（慎用）
pthread_attr_setstackaddr()	已弃用
pthread_attr_setstacksize()	设置栈大小

第三步：创建线程，使用你刚才设置的属性

```
pthread_create(&id,&myattr,任务函数,任务函数的参数);
```

第四步：销毁属性

```
int pthread_attr_destroy(pthread_attr_t *attr);
```

3.以设置线程的分离属性作为例子，帮助大家理解线程属性设置步骤

分离属性分为两种：

可分离：线程不需要主函数去回收，想回收也回收不了(pthread_join不会阻塞)

不可分离：线程默认是不可分离的，线程退出的时候，主函数需要回收它，如果不回收就变成了孤儿线程

```
int pthread_attr_setdetachstate(pthread_attr_t *attr, int detachstate);
```

参数：detachstate --》PTHREAD_CREATE_DETACHED 可分离

PTHREAD_CREATE_JOINABLE 不可分离(线程默认是不可分离的)

二.线程互斥锁的使用

1.互斥锁的作用

协调多个线程对于共享资源的访问，保证任意时刻只有一个线程在访问共享资源(临界区资源)

2.互斥锁的原理和特点

原理: 通过上锁和解锁操作, 实现任意时刻只有一个线程在访问共享资源

某个线程如果对共享资源上锁了, 那么其他线程都无法使用该共享资源, 只有等到解锁以后其他线程才能使用共享资源

特点:

多个线程使用同一把锁, 谁先上锁, 那么后面上锁的线程都会阻塞

死锁:

定义: 互斥锁上完锁以后, 没有对应的解锁操作, 导致这把锁任何线程都无法使用

产生死锁常见的原因:

原因一: 程序员粗心大意, 忘记解锁

解决方法: 把解锁操作代码加上即可

原因二: 线程刚上锁, 还没有来得及解锁就被取消了

解决方法: 可以把线程设置成不能被取消

3.互斥锁相关的接口函数

(1)初始化互斥锁

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr);
```

参数: mutex --》互斥锁变量

mutexattr --》互斥锁的属性, 一般设置为NULL, 表示使用系统默认的属性

(2)使用互斥锁--》两种操作上锁和解锁

上锁

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

解锁

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

(3)互斥锁的销毁

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

三.练习作业

1.多个线程使用不同的锁, 会不会阻塞?

互斥锁--》字面上理解就是互相排斥(多个线程争抢同一个锁才会互相排斥)

2.线程上了锁以后, 忘记解锁, 会怎么样? --》会出现死锁